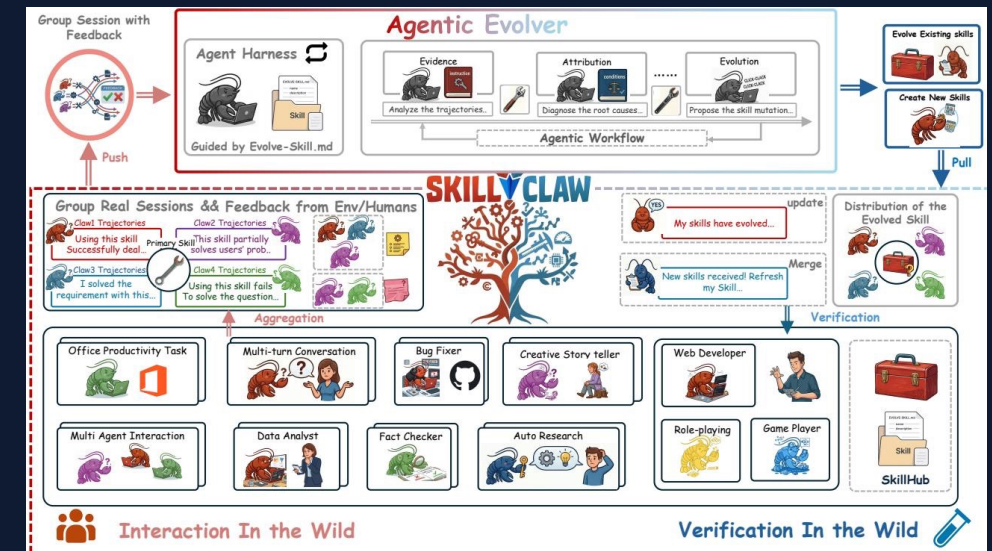


SkillClaw

Collective Skill Evolution for Multi-User LLM Agent Ecosystems

Let skills evolve collectively with agentic evolver

arxiv.org/abs/2604.08377



Agent Skills Remain Static After Deployment

Users rediscover solutions independently — the system never learns

Current Paradigm: Static Skills

Skills are manually installed from a centralized hub

Solutions discovered during interaction do not persist beyond individual sessions

Same failure patterns recur across users and over time

Each user must rediscover solutions independently

Memory-based methods: store trajectories but remain instance-specific, hard to generalize

Skill-based methods: compress experience into instructions but treat the library as a static resource

Desired Paradigm: Evolving Skills

Skills should improve from runtime experience

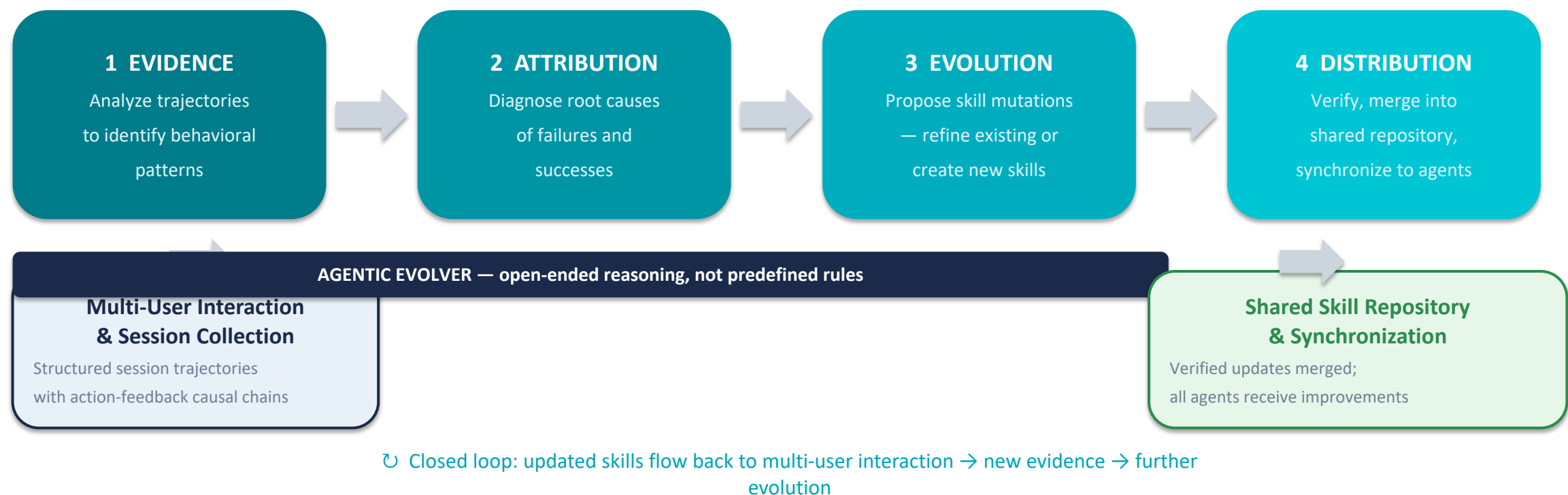
Cross-user knowledge accumulates at the system level

Failure patterns in one session inform future behavior

Success patterns propagate to all users automatically

SkillClaw closes this gap: collective, automatic, agentic skill evolution

Closed-Loop Pipeline: From Multi-User Interaction to Shared Skill Evolution



Compatible Agent Systems

OpenClaw • CoPaw • IronClaw • PicoClaw • ZeroClaw • NanoClaw • NemoClaw

Evaluation on WildClawBench with Qwen3-Max backbone demonstrates substantial improvements across tasks

Agentic Evolver: Evidence → Attribution → Evolution in Three Stages

01

EVIDENCE

Analyze Trajectories

- Aggregate skill-specific session groups from multiple users
- Preserve full action–feedback causal chains
- Identify behavioral patterns: where skills succeed vs. fail
- Cross-user diversity reveals skill behavioral boundaries



02

ATTRIBUTION

Diagnose Root Causes

- Distinguish generalizable failures from idiosyncratic edge cases
- Map outcomes to specific skill instructions or missing capabilities
- Correlate failure modes across independent user sessions
- Single user rarely generates enough signal for reliable attribution



03

EVOLUTION

Propose Skill Mutations

- Refine existing skill instructions based on attributed causes
- Create new skills when gaps are identified
- Candidate mutations verified before merging
- Open-ended agent reasoning — not predefined update rules

Key insight: The evolver is an autonomous agent performing open-ended reasoning over interaction evidence — not a rule-based updater.

Cross-User Trajectory Aggregation Exposes Skill Behavioral Boundaries

Why aggregation matters

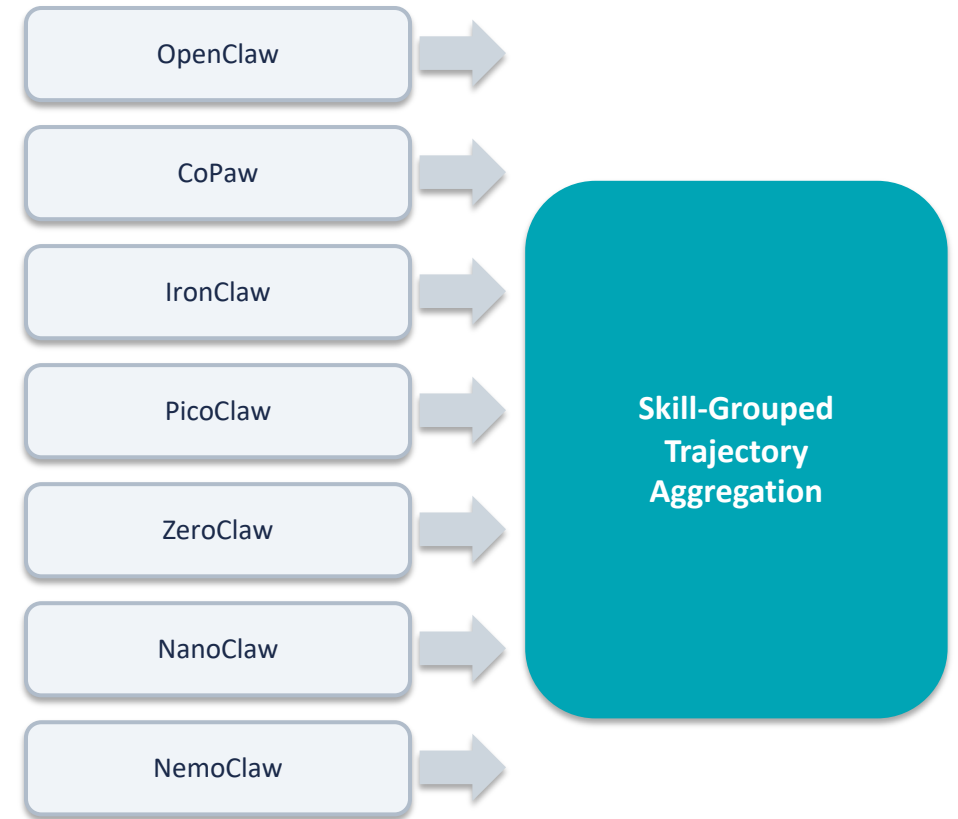
Different users exercising the same skill under diverse contexts produce complementary views of that skill's behavioral boundary

Reveals both conditions under which a skill works and those under which it breaks

A single user rarely generates enough signal to separate a generalizable improvement from an idiosyncratic fix

Group session aggregation consolidates signals across users

Trajectories are grouped by referenced skills, forming a shared evidence base



→ Feeds the Agentic Evolver (Evidence stage)

Case Study: Multimodal Creative Pipeline Skill — 6 Days of Evolution

Only 1 of 6 candidates accepted — conservative verification prevents regressions

Day	Candidate Skill	Validator	Next-Day Action
1	validate-tmp-workspace-inputs	Accept	Upgrade to new best pool
2	multimodal-input-validation-and-setup	Reject	Keep current best pool
3	multimodal-creative-task-pipeline	Reject	Keep current best pool
4	multimodal-creative-task-pipeline (impr.)	Reject	Keep current best pool
5	multimodal-creative-task-pipeline (impr.) + validate-required-input-files	Reject	Keep current best pool
6	multimodal-creative-task-pipeline (cand.)	Reject	Not admitted to next cycle

Day 1 accepted candidate addressed a real gap (input validation); Days 2–6 candidates were rejected as they did not improve over the existing best pool.

Case Study: Git Push Auth-Fallback Skill — Iterative Refinement Over 6 Days

3 of 6 days yielded accepted updates — successful iterative refinement before plateau

Day	Candidate Skill / Change	Validator	Next-Day Action
1	git-push-with-auth-fallback Safe fallback instead of blocking on push failure	Accept	Add to Safety best pool
2	git-push-with-auth-fallback Unified patch/bundle filenames, reduced inconsistency	Accept	Keep updated best pool
3	git-push-with-auth-fallback + git-clone-to-directory Auth-alternative paths & secrets audit; fixed subshell pitfalls	Accept	Keep current best pool
4	(none) — same-pool retest Validator confirmed current pool as best	Accept	Continue same best pool
5	git-push-with-auth-fallback Added "push hang treated as auth failure"; no improvement	Reject	Keep current best pool
6	git-push-with-auth-fallback Added identity config & filename consistency; no improvement	Reject	Not admitted to next cycle

Days 1–3 show meaningful improvements accepted; Day 4 confirms stability; Days 5–6 candidates rejected — the skill reached a plateau.

Key Takeaways: Three Properties That Distinguish SkillClaw

1

Collective Evolution

Knowledge from individual interactions contributes to a shared, continuously improving skill ecosystem

2

Fully Automatic

Skill evolution is driven by runtime interaction — no manual curation or explicit user intervention needed

3

Agentic Evolution Paradigm

Skill updates produced through open-ended reasoning over evidence, not predefined update rules

Implications for Continuously Improving Agent Systems

Agent ecosystems can improve from usage without human-in-the-loop curation

Conservative verification ensures regressions are prevented — only validated mutations propagate

Cross-user diversity is the fuel: more users → richer evidence → better evolution

Applicable across Claw-style systems (OpenClaw, CoPaw, IronClaw, PicoClaw, ZeroClaw, NanoClaw, NemoClaw)